Q1. Create a table named students with fields:

- stdid INT PRIMARY KEY
- stdname VARCHAR(50)
- age INT
- city VARCHAR(50)

```
mysql> create table students (
    -> stdid INT PRIMARY KEY,
    -> stdname VARCHAR(50),
    -> age INT,
    -> city VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.12 sec)
```

Q2. Insert the following records into the students table:

```
mysql> insert into students (stdid, stdname, age, city) VALUES
    -> ('1', 'Rohan', '20', 'pune'),
    -> ('2', 'Meera', '22', 'Mumbai'),
    -> ('3', 'Arjun', '21', 'Delhi'),
    -> ('4', 'Kavya', '23', 'Pune'),
    -> ('5', 'Neha', '22', 'Kolkata');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Q3. Display all student records.

```
mysql> select * from students;
+-------+---------+------+---------+
| stdid | stdname | age  | city    |
+-------+---------+------+---------+
|     1 | Rohan   |   20 | pune    |
|     2 | Meera   |   22 | Mumbai  |
|     3 | Arjun   |   21 | Delhi   |
|     4 | Kavya   |   23 | Pune    |
|     5 | Neha    |   22 | Kolkata |
+-------+---------+------+---------+
5 rows in set (0.00 sec)
```

Q4. Display only the name and age of all students.

```
mysql> select stdname, age from students;
+---------+------+
| stdname | age  |
+---------+------+
| Rohan   |   20 |
| Meera   |   22 |
| Arjun   |   21 |
| Kavya   |   23 |
| Neha    |   22 |
+---------+------+
5 rows in set (0.00 sec)
```

Q5. Display students who are from Pune.

```
mysql> select * from students where city = 'Pune';
+-------+---------+------+------+
| stdid | stdname | age  | city |
+-------+---------+------+------+
|     1 | Rohan   |   20 | pune |
|     4 | Kavya   |   23 | Pune |
+-------+---------+------+------+
2 rows in set (0.01 sec)
```

Q6. Display students whose age is greater than 21.

```
mysql> select * from students where age>21;
+-------+---------+------+---------+
| stdid | stdname | age  | city    |
+-------+---------+------+---------+
|     2 | Meera   |   22 | Mumbai  |
|     4 | Kavya   |   23 | Pune    |
|     5 | Neha    |   22 | Kolkata |
+-------+---------+------+---------+
3 rows in set (0.01 sec)
```

Q7. Display students in descending order of age.

```
mysql> select * from students ORDER BY age DESC;
+-------+---------+------+---------+
| stdid | stdname | age  | city    |
+-------+---------+------+---------+
|     4 | Kavya   |   23 | Pune    |
|     2 | Meera   |   22 | Mumbai  |
|     5 | Neha    |   22 | Kolkata |
|     3 | Arjun   |   21 | Delhi   |
|     1 | Rohan   |   20 | pune    |
+-------+---------+------+---------+
5 rows in set (0.00 sec)
```

Q8. Count how many students belong to each city. (Use GROUP BY)

```
mysql> select city, COUNT(*) AS total_students
    -> from students
    -> group by city;
+---------+----------------+
| city    | total_students |
+---------+----------------+
| pune    |              2 |
| Mumbai  |              1 |
| Delhi   |              1 |
| Kolkata |              1 |
+---------+----------------+
4 rows in set (0.01 sec)
```

Q9. Display students whose name starts with ‘K’. (Use LIKE)

```
mysql> select * from students
    -> where stdname LIKE 'K%';
+-------+---------+------+------+
| stdid | stdname | age  | city |
+-------+---------+------+------+
|     4 | Kavya   |   23 | Pune |
+-------+---------+------+------+
1 row in set (0.01 sec)
```

Q10. Delete student whose stdid = 5.

```
mysql> delete from students where stdid = 5;
Query OK, 1 row affected (0.02 sec)

mysql> select * from students;
+-------+---------+------+--------+
| stdid | stdname | age  | city   |
+-------+---------+------+--------+
|     1 | Rohan   |   20 | pune   |
|     2 | Meera   |   22 | Mumbai |
|     3 | Arjun   |   21 | Delhi  |
|     4 | Kavya   |   23 | Pune   |
+-------+---------+------+--------+
4 rows in set (0.00 sec)
```

Q11. Display student name and marks of only those students who have matching IDs in both tables.
(Students without marks should not appear.)

```
mysql> select students.stdname, marks.marks from students
    -> inner join marks
    -> on students.stdid = marks.stdid;
+---------+-------+
| stdname | marks |
+---------+-------+
| Rohan   |    88 |
| Meera   |    76 |
| Arjun   |    92 |
+---------+-------+
3 rows in set (0.01 sec)
```

Q12. Display all students and their marks.
(If marks not available, show NULL.)

```
mysql> select students.stdname, marks.marks
    -> from students
    -> left join marks
    -> on students.stdid = marks.stdid;
+---------+-------+
| stdname | marks |
+---------+-------+
| Rohan   |    88 |
| Meera   |    76 |
| Arjun   |    92 |
| Kavya   |  NULL |
+---------+-------+
4 rows in set (0.01 sec)
```

Q13. Display all marks records along with student names. (If student doesn't exist in students table, show NULL.)

```
mysql> select marks.subject, students.stdname,
    -> marks.stdid, marks.marks
    -> from students
    -> right join marks
    -> on students.stdid = marks.stdid;
+---------+---------+-------+-------+
| subject | stdname | stdid | marks |
+---------+---------+-------+-------+
| Maths   | Rohan   |     1 |    88 |
| Maths   | Meera   |     2 |    76 |
| Maths   | Arjun   |     3 |    92 |
| Maths   | NULL    |     5 |    67 |
+---------+---------+-------+-------+
4 rows in set (0.00 sec)
```

Q14. Display all possible combinations of students and subjects.
(Use CROSS JOIN between students and marks table to show every pair.)

```
mysql> select students.stdname, marks.subject
    -> from students
    -> cross join marks;
+---------+---------+
| stdname | subject |
+---------+---------+
| Rohan   | Maths   |
| Rohan   | Maths   |
| Rohan   | Maths   |
| Rohan   | Maths   |
| Meera   | Maths   |
| Meera   | Maths   |
| Meera   | Maths   |
| Meera   | Maths   |
| Arjun   | Maths   |
| Arjun   | Maths   |
| Arjun   | Maths   |
| Arjun   | Maths   |
| Kavya   | Maths   |
| Kavya   | Maths   |
| Kavya   | Maths   |
| Kavya   | Maths   |
+---------+---------+
16 rows in set (0.00 sec)
```

Q15. Using INNER JOIN, display students who scored more than 80.

```
mysql> select s.stdname, m.subject, m.marks
    -> from students s
    -> inner join marks m
    -> on s.stdid = m.stdid
    -> where m.marks > 80;
+---------+---------+-------+
| stdname | subject | marks |
+---------+---------+-------+
| Rohan   | Maths   |    88 |
| Arjun   | Maths   |    92 |
+---------+---------+-------+
2 rows in set (0.01 sec)
```